

Analyse v8 → v9 — Brent C/mpfr vs Illinois

hprzeta --- Riemann Lab

12 juin 2026

Date : 2026-06-12

Branche : Riemann_Lab_C

Auteur : hprzeta

1. Contexte

v8 (11 juin 2026) avait atteint le **plancher hardware** du i7-7500U :

- `prec_full` = 80 bits → gain $\times 1.06$ vs v7 (116 bits) → marginal
- `W` = 8 workers → $\times 0.99$ vs `W`=4 → context-switching dual-core HT
- Référence v7 T=100k : **30.9 min, 74.49 z/s** (avec turbo)

Seul levier algorithmique restant : **réduire le nombre d'itérations** d'affinage.

2. Hypothèse v9

La méthode de Brent (Van Wijngaarden-Dekker-Brent) a un **ordre de convergence** ~ 1.84 vs Illinois-modifié ~ 1.44 . Chaque itération coûte **1 seul appel `Z_rs_mpfr_ntermes`** (identique à Illinois). Gain attendu : $\times 1.2$ - 1.4 local sur les itérations → $\times 1.2$ global (gain asymptotique réduit par la détection, turing, etc.).

Méthode	Ordre	Évals/iter	Gain iter	Verdict
Illinois	1.44	1	baseline	✓ v8
Brent	~ 1.84	1	$\sim \times 1.7$	✓ v9

3. Implémentation

`brent_mpfr.c`

Fichier : `src/calculs/optimisation/c_modules/brent_mpfr.c`

Algorithme IQI + sécante + bisection avec 5 conditions de sécurité :

`cond1` : $s \notin [(3a+b)/4, b]$ → force la convergence vers `b`
`cond2` : `mflag` && $|s-b| \geq |b-c|/2$ → pas assez de progrès
`cond3` : !`mflag` && $|s-b| \geq |c-d|/2$ → idem
`cond4` : `mflag` && $|b-c| < \text{tol}$ → stagnation

cond5 : !mflag && |c-d| < tol → stagnation

Invariant Brent : $|f(b)| \leq |f(a)|$ maintenu à chaque itération par swap.

Paramètres v9

```
ITER_SWITCH = 3  # Brent : 3 itérations phase1 suffisent (vs 8 Illinois)
MAX_ITER    = 50 # identique à v8 par sécurité
PREC_FAST   = 64 # phase 1 – 1 limbe SIMD (inchangé)
PREC_FULL   = 80 # phase 2 – optimal mesuré (inchangé)
```

Compilation

```
cd src/calculs/optimisation/c_modules
make illinois_mpfr.so
# GCC compile illinois_mpfr.c brent_mpfr.c ensemble → même .so
```

4. Benchmarks

4.1 benchmark_methodes.py — Illinois (v8 référence)

500 zéros distribués dans [1000, 100000] depuis le CSV T=100k v7.

Métrique	Valeur
ms/appel moyen	128.9 ms
ms/appel médiane	118.4 ms
ms/appel t=1k	6.76 ms
ms/appel t=10k	53.28 ms
ms/appel t=50k	106.43 ms
ms/appel t=77k	192.56 ms
Iter phase1 (moy)	8.0 (ITER_SWITCH fixe)
Iter phase2 (moy Python)	39.7
Iter totales (moy)	47.7

Observation clé : le coût croît fortement avec t (proportionnel à $N_{full} = \text{floor}(\sqrt{t/2\pi}) \times \text{coût MPFR}$). Le benchmark précédent (benchmark_v8.py) ne mesurait que $t \in [1000, 3000] \rightarrow 0.416$ ms biaisé bas.

4.2 test_brent.py — Comparaison Brent vs Illinois (100 zéros, $t \in [1000, 50000]$)

Méthode	ms/appel moy	iter moy	gain
Illinois (v8)	12.0 ms	45.1	baseline
Brent (v9)	6.75 ms	27.2	×1.78

4.3 Validation T=10k v9

Indicateur	v8	v9	Δ
Zéros	10 142	10 142	=
Durée	17.2 s	16.6 s	-3.5%
Vitesse	591 z/s	609 z/s	+3%
Turing	COMPLET	COMPLET	=
LMFDB	19/20	19/20	=
brent_C %	—	98.6%	—

Note : à petit t ($T=10k$), le gain Brent est modeste (+3%) car N_{full} est petit (faible coût/iter). Le gain $\times 1.78$ du benchmark ($t \in [1000, 50000]$) devrait être mieux représenté à $T=100k$ où la majorité des zéros est à grand t .

5. Limitation RS connue ($t < 300$)

`brent_refine_adaptive` utilise `Z_rs_mpfr_ntermes` dont l'erreur intrinsèque est $O(t^{-3/4})$.

Pour $t < 300$: $N_{full} < 7$ termes $\rightarrow$ erreur RS $\sim 1e-2 \rightarrow$ imprécis.

Identique à Illinois v8 — pour $t < 300$, v9 utilise `mpmath.findroot` (1.4% des zéros).

6. Décision go/no-go

Critère	Seuil	v9	Verdict
Gain ms/appel (benchmark)	$\geq \times 1.05$	$\times 1.78$	✓ GO
$T=10k$ zéros manquants	0	0	✓ GO
$T=10k$ Turing	COMPLET	COMPLET	✓ GO
$T=10k$ LMFDB	$\geq 19/20$	19/20	✓ GO

Décision : GO pour $T=100k$ v9.

7. Questions ouvertes

- Le gain réel sur $T=100k$ sera mesuré fin de run (estimé ~ 43 min sans turbo, ~ 35 min avec).
- Si gain $< \times 1.1$: explorer `Arb.acb_dirichlet_hardy_z` (TÂCHE 6 — optionnelle).
- Amélioration possible : pré-calculer `mpfr_const_pi` une seule fois par worker (évite N_{iter} recomputations) $\rightarrow$ gain supplémentaire possible.

analyse_problemes_v8_v9.md · Riemann_Lab.wiki/master · 2026-06-12 · ~ 120 lignes